

Maximal Format-Free Data Repair

Zijian Luo
University of Sydney
Sydney, Australia
lzij5923@uni.sydney.edu.au

Xi Wu
University of Sydney
Sydney, Australia
xi.wu@sydney.edu.au

Hong Jin Kang
University of Sydney
Sydney, Australia
hongjin.kang@sydney.edu.au

Alan Fekete
University of Sydney
Sydney, Australia
alan.fekete@sydney.edu.au

Rahul Gopinath
University of Sydney
Sydney, Australia
rahul.gopinath@sydney.edu.au

ABSTRACT

Modern data-processing pipelines rely on input records conforming to strict format specifications. In practice, however, data corruption can occur at numerous stages including data-entry error, corruption during input processing and retransmission, inconsistent formatting, and incompatible specifications. Such corrupted data can result in loss of records, reducing the accuracy of processing.

Rather than discarding corrupted records and losing valuable information, one can attempt to repair the data. Data-repair solutions such as *regular expression based repair* and *error-correcting parsers* require a specification to perform structural repairs.

Specification-free techniques such as *ddmax* and ϵ REPAIR are limited in repair operations, repair location, and require specific parser properties that are often unavailable.

To tackle this challenge, we introduce β MAX, a novel format-free data repair algorithm optimal with respect to the provided example data, with maximal data-recovery and minimal parser constraints.

Despite requiring less information than ϵ REPAIR, β MAX repairs 83% of all corrupt records—1.77 $\times$ the rate achieved by ϵ REPAIR—while using 27.7 $\times$ fewer oracle calls.

CCS CONCEPTS

• **Information systems** $\rightarrow$ Data cleaning; • **Theory of computation** $\rightarrow$ Regular languages; • **Software and its engineering** $\rightarrow$ Software testing and debugging.

KEYWORDS

Input repair, Error correction, Data cleansing

ACM Reference Format:

Zijian Luo, Xi Wu, Hong Jin Kang, Alan Fekete, and Rahul Gopinath. 2026. Maximal Format-Free Data Repair. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3832783.3834354>

1 INTRODUCTION

While processing data collated from diverse sources, one must often contend with malformed data. This corruption could be artifacts of inconsistent processing [24, 44, 51], hardware failures [26], truncated network transmission [12, 49], or even human errors in data entry [32, 50]. Even within a single organization, CSV file formats can be inconsistent, with dates and times frequently containing incompatible delimiters [29]. Over 99% of spreadsheets contain non-validated columns [20], with over 35% of the columns representing data as simply of *string* type, which can result in inconsistencies.

In a data processing pipeline, such parser inconsistencies can cause record processing to fail. Even if the inconsistency is in a single field in a tabular data structure, automatic recovery can be infeasible [46, 48], and manual repair can be costly and error-prone [40]. Hence, data records containing errors are often *cleansed* from the pipeline [25], resulting in severe data loss.

Hence, automatic data repair techniques are needed. Format-based repair [1, 15, 18, 19, 30, 35] can be applied if a format is available. However, format specifications are often rare, especially for one-off application-specific formats that are typically used in the processing of tabular data such as CSV, Excel, TSV etc.

Researchers have explored specification-free alternatives to data repair. For example, *ddmax* [31] can fix malformed data records provided three constraints are satisfied: (1) a parser that can recognize valid ($\checkmark$) and invalid ($\times$) inputs, (2) a starting minimal subsequence of the input ($\emptyset$) that is accepted as a valid parse, and (3) the ability to add fragments (δ s) to this input, resulting in progressively larger *parsable inputs* (Ξ_i) called *waypoints*. Given a data format that conforms to these requirements and a corresponding parser, starting from Ξ_0 which is $\emptyset^\checkmark$, *ddmax* will iteratively identify chunks (δ_i) from the given (malformed) input that can be added to Ξ while retaining parsability resulting in a Ξ_{max} representing maximal data recovery from the original input. That is, data repair with *ddmax* requires a conforming data format that has an empty valid parse ($\emptyset^\checkmark$), parsable waypoints ($\Xi_i^\checkmark$), and a corresponding parser. However, data formats such as addresses, dates, times, zip codes, and telephone numbers common in tabular data do not have a *minimal* valid input that correspond to $\emptyset$. Furthermore, *ddmax* is limited to simply *deleting* any input fragment it deems as part of the corruption, exacerbating data loss when corruption is misidentified.

ϵ REPAIR is another algorithm for data repair with a different tradeoff. Unlike *ddmax*, ϵ REPAIR does not require a starting empty sequence $\emptyset$, nor parsable waypoints Ξ_i . Instead, it assumes that

the *location of the corruption* can be identified from parser feedback. This allows ϵ REPAIR to repair inputs even when valid minimal sequence ($\emptyset$) or valid waypoints (Ξ_i) are unavailable. Furthermore, ϵ REPAIR utilizes all corrections—*deletion*, *insertion*, and *replacement*—achieving better repair.

However, ϵ REPAIR has two limitations: (1) It requires precise error location, which is often unavailable. (2) It conflates the location of minimal repair with the error location returned by the parser, which may not lead to minimal repairs.

To address these issues, we propose β MAX, a novel approach for automatic data repair. The key innovation in β MAX is to recognize that in most data-repair settings, there will be numerous inputs that are accepted by the parser program, and a few that are corrupted and hence, rejected. This provides us with a set of valid and invalid examples to form an initial approximate specification [42], which is then iteratively refined. This approximate specification is used in conjunction with the error correcting parser [2] to produce an initial *repair* of a given corrupt input. The repair, if not accepted by the parser, is then used to refine the approximate specification, producing closer repairs iteratively until a repair is accepted.

β MAX has two key advantages over ϵ REPAIR: (1) Being independent of parser characteristics, it can be applied to any data format that has a simple validator. (2) Similar to numerical data-imputation, the repair provided is structurally closest to available samples, reducing impact of a potentially incorrect repair on the data set. β MAX focuses on repairing individual fields in data records—dates, times, network addresses—which are typically regular languages validated by matchers offering only accept/reject feedback.

Although β MAX operates with less information than ϵ REPAIR, β MAX performs 1.77 $\times$ better than ϵ REPAIR, repairing 83% of all corrupt records, while using 27.7 $\times$ fewer oracle calls.

We make the following contributions:

- A novel specification-free data-repair algorithm expecting only a parser that can accept ($\checkmark$) or reject ($\times$) a given input.
- Improvement in data-repair quality and performance compared to the state-of-the-art for regular formats.
- An empirical comparison of ϵ REPAIR and β MAX.

2 CHALLENGES IN DATA REPAIR

The problem of data repair is the following: Given a corrupt string, find the best repair such that the maximal amount of original data is saved [38]. For example, consider the JSON fragment below:

```
{ "name": "John Doe", "email": "john@doe.com john@doe.org",
  "phone": "2 8627 1444", "date": "2025/03/01",
  "web": "http://f0f.:]:80 http://[f0fhost.com:80" }
```

After the first level of parsing by a JSON parser, the individual fields will undergo further processing, for example, to validate *email*, *phone*, and *date*, each of which are specified as strings. Typically, this validation will be done using regular expressions or small custom functions. In this example, there is a problem: The *email* field contains two emails rather than the expected single email. Perhaps the emails should have been separated with ‘;’ rather than a space. This is a relatively common error that can cause the parser to fail on this field, and reject the entire record. To recover the needed data, we need to repair this field. A similar issue applies to

phone where spaces are not allowed. Similarly, the date format may be parsed by ISO8601 [28], which requires ‘-’ as the delimiter.

We consider each state-of-the-art data repair methods in turn.

2.1 Data Repair with *ddmax*

The *ddmax* algorithm is based on *ddmin*, and works as follows: $\checkmark$ denotes parser acceptance, $\times$ denotes rejection.

- (1) *ddmax* begins with an empty valid input ($\emptyset$). Let us call this input Ξ_0 . The input can also be a minimal starting value. Such minimal values should conform to two requirements: (1) It should be acceptable as the smallest *valid* input for the given format. For example, $\Xi_0 = \{\}$ $\checkmark$ is a reasonable starting input for JSON objects. (2) It should be producible by deleting fragments from the original input. If not, the input cannot be repaired by *ddmax*.
- (2) The *ddmax* algorithm then identifies fragments (named δ_i) in the original input that can be added to Ξ_i such that $\Xi_{i+1} = \Xi_i + \delta_i$ $\checkmark$. Any δ_i such that $\Xi_i + \delta_i$ $\times$ is rejected. This process continues until all δ_i in the input is exhausted.
- (3) This produces Ξ_{max} $\checkmark$ representing the maximal repair.

Considering the malformed email field in the corrupt record, we find three impediments to applying *ddmax*. (1) There is no simple equivalent for an empty field ($\emptyset$) $\checkmark$ for *phone* and *date*, each of which are fixed width fields. Hence, *ddmax* cannot be applied on such fields. (2) Even in variable width fields where one can find an empty field such as *email* which satisfies the parser (for example, $\emptyset = @$) it can be non-obvious how to obtain that input, and in some cases, there may be multiple such inputs. For example, it is non-obvious what the minimal starting input should be for repairing $\{ "abc" : [] \}$. Doing this for thousands of diverse fields can be tedious and error-prone. (3) *ddmax* uses *deletion* as the only repair action. Deletion is insufficient to repair fixed width fields with data loss.

Hence, *ddmax* is unsuitable for data repair under such conditions. Let us now examine how ϵ REPAIR fares on such malformed inputs.

2.2 Data Repair with ϵ REPAIR

ϵ REPAIR works as follows: Let $\checkmark$ indicate an input accepted by the parser as before. However, the parser is also expected to distinguish when the *input* is a valid *prefix* of some valid string. For example, $\{ "key" : \}$ is a valid JSON prefix, and is indicated by $\triangleright$. If the given text is not a valid prefix, for example, $\{ \{ "key" : \}$ which does not have a valid completion, it is indicated by $\times$. The $\triangleright$ signal can also be inferred if the parser reports the syntax error location.

Given a malformed input and a parser that can distinguish $\triangleright$ from $\times$, ϵ REPAIR proceeds as follows:

- (1) **Boundary search.** ϵ REPAIR begins by locating the position of the syntax error (called the *parse boundary*) using binary search. This boundary is used to initialize the thread queue with a single empty repair thread.
- (2) **Apply Repair.** From any active repair thread, ϵ REPAIR performs either a deletion or an insertion at the parse boundary, producing new repair threads:
 - (a) A repair thread where a single character or token is deleted from the input at the error location, allowing extending the parse boundary using the original data.

- (b) Repair threads where a character is inserted at the error location for each character in the alphabet that yields an $\triangleright$ response, enabling the parse boundary to be extended using the original data.
- (3) **Extend Boundary.** For each repair thread, the parse boundary is extended by appending remaining characters. Non-extendable threads are discarded.
- (4) **Select Threads.** The algorithm selects the highest-priority repair thread from the priority queue (ordered by preserved data) and repeats from step 2. Duplicate threads are eliminated, retaining only those with the fewest repairs.
- (5) **Check Completion.** When the parser signals (i.e. $\checkmark$) that a repair thread is complete, the process terminates and the repaired input is returned.

While ϵ REPAIR handles structurally rich formats, its generality is limited when corruption occurs in fields validated by ad hoc matchers or when parser feedback is unreliable.

Limited Usefulness of Parser Feedback. Corruptions in fields such as *email*, *phone*, or *date* are typically checked using regular expressions, and such validators offer only a match/non-match result. Without information about whether the input is merely incomplete or fundamentally incorrect, ϵ REPAIR cannot infer a boundary to guide local edits. As a consequence, these common scalar fields fall entirely outside its repairable scope.

Sensitivity to Error-Boundary Placement. ϵ REPAIR assumes that the parser’s error location reliably indicates where repairs should occur. This assumption can be misleading. For example, in `http://f00f:•:]:80`, the local boundary reported after `f00f:` directs ϵ REPAIR toward deleting `]:`, whereas the optimal fix is to insert `[` before `f00f`. Similarly, in `http://[f0f•host.com:80`, deleting the stray `[` is the correct solution. Yet it lies outside the parser-indicated region. In such cases, ϵ REPAIR’s locality bias prevents it from recovering globally minimal repairs.

These examples illustrate a broader pattern: methods that depend on parse-boundary signals remain vulnerable to coarse, incomplete, or unhelpful parser diagnostics. This dependency makes it difficult to extend ϵ REPAIR to formats validated only by regular expressions, or to parsers whose feedback cannot reliably guide structural edits.

These limitations point toward a natural design direction: if feedback cannot be trusted, then a repair technique must avoid relying on parser characteristics in the first place.

The next section introduces a complementary approach, called β MAX, that removes this dependency. Unlike *ddmax* and ϵ REPAIR, β MAX grounds its repairs in provided example data rather than parser feedback, producing repairs that are optimal with respect to an iteratively refined grammar.

3 RECORD REPAIR WITH BMAX

We first define a few terms used in this section.

- oracle** A blackbox that accepts *string* inputs and returns $\times$ or $\checkmark$.
- alphabet** Symbols in the input language of the *oracle*.
- string** An ordered sequence of input symbols. A string is *valid*, or *accepted* if the *oracle* returns $\checkmark$ when given this string, and it is *invalid*, or *rejected* if the *oracle* returns $\times$.
- grammar** A grammar defines the input specification for an oracle.

Similar to specification-based data-repair [1, 15, 43], β MAX assumes that the corrupted data originally conformed to some specification. Following numeric data imputation principles, adapted for non-numeric, structured data, β MAX assumes that the original data is similar to the provided valid examples, and the repair algorithm’s task is to produce a repair that is closest to these examples.

Consequently, β MAX requires valid example data and, similar to both *ddmax* and ϵ REPAIR, assumes that there exists an oracle that will respond $\checkmark$ to valid inputs, and $\times$ to invalid inputs. Note that β MAX is designed for, and leverages *regular-language* inference.

Let us start with an oracle that accepts the *hostname* (RFC952 [23]). The β MAX algorithm starts with a few examples: (1) $a^{\checkmark}$ (2) $ac^{\checkmark}$ (3) $b1e^{\checkmark}$ (4) $b1f^{\checkmark}$. An invalid string (5) $1f^{\times}$ needs repair.

The top-level β MAX algorithm is given in Algorithm 4. We describe each step in the subsections that follow.

3.1 Prefix-Tree Acceptor

β MAX begins by creating a prefix-tree acceptor automata (PTA) that corresponds to the given valid strings (Figure 1). Creating such a PTA is the first step in grammar inference from labeled data [37].

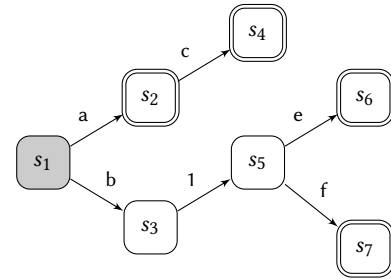


Figure 1: Prefix tree automata

$$\begin{aligned}
 \langle s_1 \rangle &\rightarrow a \langle s_2 \rangle \mid b \langle s_3 \rangle \\
 \langle s_2 \rangle &\rightarrow c \langle s_4 \rangle \mid \epsilon \\
 \langle s_3 \rangle &\rightarrow 1 \langle s_5 \rangle \\
 \langle s_4 \rangle &\rightarrow \epsilon \\
 \langle s_5 \rangle &\rightarrow f \langle s_7 \rangle \mid e \langle s_6 \rangle \\
 \langle s_6 \rangle &\rightarrow \epsilon \\
 \langle s_7 \rangle &\rightarrow \epsilon
 \end{aligned}$$

Figure 2: The automata as grammar

Each prefix maps to a unique state, and the PTA accepts the input strings exactly, capturing the most specific behavior consistent with these strings. The states in the PTA are ordered and numbered.

An automaton can be represented as a *regular grammar*. The starting state of the automaton becomes the start symbol of the regular grammar. Each state is represented by a nonterminal symbol in the grammar, and each transition is represented by a terminal symbol. The transitions from a state are represented by production rules in the equivalent grammar, starting with the transition symbol, and ending with the nonterminal symbol corresponding to the next state. The accepting state is represented by a nonterminal

symbol that has empty string (ϵ) as the production rule. The regular grammar for the PTA in Figure 1 is given in Figure 2.

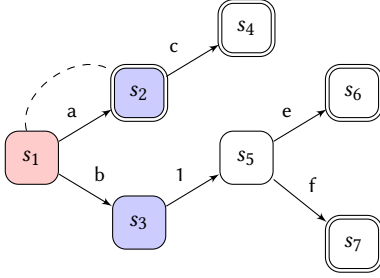


Figure 3: After step (1), (2), and (3)

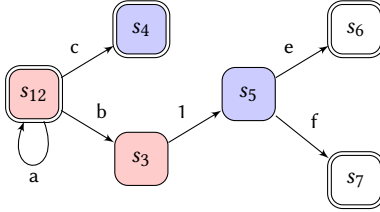


Figure 4: After step (4), (5), (2)

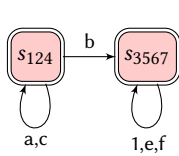


Figure 5: Automata A_1 without blue states

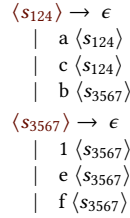


Figure 6: G_1 grammar

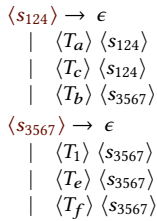


Figure 7: G_1^{cover} grammar

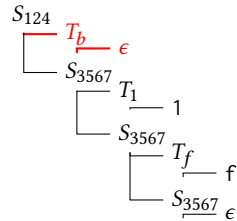


Figure 8: $1fX$ Parse tree.

Let us call this grammar G_0 . From now on, we will assume that the automaton is represented as a regular grammar.

Augmenting Invalid Examples with Neighborhood Exploration. In traditional algorithms for inferring languages from labeled data [42], the labeled valid strings determine the effectiveness of *recall* of the final grammar. On the other hand, the effectiveness of *precision* is determined by the labeled invalid strings. We found that a single invalid string (the string to be repaired) was insufficient

to generate grammars with sufficient precision and we needed additional invalid strings. While it is easy to generate invalid strings, we found that the *quality* of such strings was important. An invalid string is of high quality if it is very close to validity. That is, at most one or two symbols away from a valid string. Such invalid strings serve to constrain the generalization process, and hence improve the *precision* of the automaton produced.

To produce such strings, we mutated the PTA one transition at a time. Then, we produced strings that contained that transition. That is, if there are n transitions in the PTA, this process produces n new PTAs, from which we produce a new set of strings that differ from the original by exactly one character. Next, we check the strings generated by these new PTAs. If the string from the newly induced transition was rejected, it is added to the invalid examples. We call this process the *neighborhood exploration*, or *NE* for short.

In our case, our example set is extended by (6) $1e^X$ and (7) $1g^X$. Note that the invalid samples do not change the PTA.

3.2 State Merging and Generalization

The next step is to detect if any pairs of states can be merged. Two states can be merged if and only if all inputs that were originally identified as invalid remain invalid. Any input that was previously valid continues to be valid when any two states in the automaton are merged. For this, we adapt the state merging algorithm from the classic RPNI blue-fringe variation [34, 42] as below.

- (1) Mark the initial state as **red** (Figure 3).
- (2) Identify all states that are reachable from the initial state in a single transition. Mark them as **blue** (Figure 3).
- (3) Pick a **blue** state in a deterministic fashion (Figure 3).
- (4) Try merging the selected **blue** state with each **red** state, one at a time. Merging two states produces a new state whose incoming and outgoing transitions are the unions of the respective transitions from both original states. Commit to the merge if the chosen merge is *consistent*, and continue with the next **blue** state in step 3 (Figure 4).
- (5) If none of the **red** states could be merged with the picked **blue** state, then color it **red**. Continue to (2) if any **blue** states exist. Otherwise, output the last automaton (Figure 5).

Incremental Refinement by Merge Replay. The initial call to STATEMERGE also records a *merge history* H consisting of all *committed* (successful) merges in their original order. When a candidate repair c_r is rejected, we add c_r as a new negative counterexample and *refine* the hypothesis by replaying only the merges in H . Any merge that would now accept a negative counterexample is skipped.

Checking Mergeability. The quality of repairs is strongly influenced by the accuracy of the grammar inferred. βMAX starts with a set of examples. The valid examples determine the *recall* of the inferred grammar, and the invalid examples determine the *precision*. To ensure that we do not merge states that should not be merged, we identify new strings that can be created by the merger of two states, and check their validity with the blackbox program.

Given a state A , any string X that passes through A can be split into X_{prefix} which is the path from the start state to A , and X_{suffix} which is the path from A to the final state. Two states A , and B , with strings X and Y passing through them, can be merged if and only if all possible strings (X_{prefix}, Y_{suffix}) are accepted by oracle. This

Algorithm 1 State Merging Algorithm

```

1: function STATEMERGE( $S^+, S^-, P$ )
2:    $A \leftarrow \text{PTA}(S^+)$ 
3:    $H \leftarrow []$  ▷ Record committed merges (in order)
4:    $\text{Red} \leftarrow \{q_0\}$ ;  $\text{Blue} \leftarrow \text{CHILDREN}(q_0)$ 
5:   while  $\text{Blue} \neq \emptyset$  do
6:      $q_b \leftarrow \text{SELECT}(\text{Blue})$ 
7:      $S_b^+ \leftarrow \{x \in S^+ \mid x \text{ passes through } q_b\}$ 
8:      $\text{merged} \leftarrow \text{false}$ 
9:     for all  $q_r \in \text{Red}$  do
10:       $S_r^+ \leftarrow \{x \in S^+ \mid x \text{ passes through } q_r\}$ 
11:      if  $\text{CANMERGE}(S_r^+, S_b^+, P)$  then
12:         $A' \leftarrow \text{MERGE}(A, q_r, q_b)$ 
13:        if  $\text{VERIFIED}(A', S^+, S^-, P)$  then
14:           $A \leftarrow A'$ 
15:           $\text{APPEND}(H, (q_r, q_b))$ 
16:           $\text{merged} \leftarrow \text{true}$ 
17:          break
18:      if not merged then
19:         $\text{Red} \leftarrow \text{Red} \cup \{q_b\}$ 
20:       $\text{Blue} \leftarrow \{c \mid s \in \text{Red}, c \in \text{children}(s), c \notin \text{Red}\}$ 
21:   return  $A, H$ 

```

Algorithm 2 Can Merge Check for Candidates

```

1: function CANMERGE( $S_i^+, S_j^+, P$ )
2:   for  $\text{sample}((x_i, x_j) \in S_i^+ \times S_j^+, K)$  do ▷ We use  $K=1$ 
3:     for  $(a, b) \in \{(x_i, x_j), (x_j, x_i)\}$  do
4:       if  $\text{CONCAT}(\text{prefix}(a), \text{suffix}(b)) \notin L(P)$  then
5:         return false
6:   return true

```

Algorithm 3 Consistency Check for Candidate Merge

```

1: function VERIFIED( $A, S^+, S^-, P$ )
2:   return  $\neg \text{any}(w \in S^- : A(w))$ 

```

Algorithm 4 β MAX Repair Algorithm

```

1: function BETAMAX( $S^+, c_0, P$ )
2:    $S^- \leftarrow \{c_0\} \cup \text{NE}(S^+, P)$ 
3:    $G, H \leftarrow \text{STATEMERGE}(S^+, S^-, P)$ 
4:   loop
5:      $c_r \leftarrow \text{MINPENALTYPARSE}(\text{COVER}(G), c_0)$ 
6:     if  $P(c_r) = \checkmark$  then return  $c_r$ 
7:      $S^- \leftarrow S^- \cup \{c_r\}$ 
8:      $G \leftarrow \text{REPLAYMERGES}(S^+, S^-, H, P)$ 

```

algorithm is given in Algorithm 2. We found a sampling strategy of $K = 1$ optimal for data recovery and repair rate (Table 11). The state-merging algorithm is given in Algorithm 1.

Checking Consistency. When merging states all invalid strings from the example set should also remain invalid after merge. For example, in the PTA from Figure 1, merging the states s_1 and s_2 results in a new automata (Figure 4) which continues to reject (5) $1f^\times$, (6) $1e^\times$, and (7) $1g^\times$. Similarly, the final automata in Figure 5 also continues to reject these inputs. This check is given in Algorithm 3.

This process results in a refined hypothesis automaton we call A_1 . The corresponding regular grammar G_1 is given in Figure 6.

3.3 Covering Grammar and Repair

The G_1 grammar from Section 3.2 is next used to generate a potential repair for our corrupted data record. This covering-grammar approach used by structural repair [1, 45] is described below.

Given a grammar G_1 , each terminal symbol x is replaced by a nonterminal symbol $\langle T_x \rangle$ with the following definition that provides tolerance for errors. Let ' Σ ' denote any character, ' Σ^+ ' one or more characters, and let ' $\Sigma_{\hat{x}}$ ' denote any character other than x . Then,

$$\langle T_x \rangle \rightarrow x \mid \Sigma^+ x \mid \Sigma_{\hat{x}} \mid \epsilon$$

Intuitively, each character x may be realized as (1) itself, (2) any non-empty sequence of characters ending in x (insertion), (3) a single character different from x (substitution), or (4) deletion. This gives us the covering grammar G^{cover} . Notably, the $\Sigma^+ x$ production permits multi-character insertions, handling two- or three-character corruptions in one step.

Next, production rules are annotated with penalties. Each use of an error-tolerant production rule such as $\Sigma^+ x$, $\Sigma_{\hat{x}}$, or ϵ incurs a positive penalty, whereas an exact match on x incurs zero penalty. We then use any context-free parser and parse the corrupted string c_0 with this grammar and choose candidate with the least penalty.

For example, parsing the corrupt input $1f^\times$ with G^{cover} produces the parse tree in Figure 8, which indicates the least number of edits to be made to the string to make it parsable by G_1 . In the parse tree (Figure 8), we replace $T_b \rightarrow \epsilon$ with b , which is the only edit needed. We apply these edits, producing the repair candidate c_1 .

This repair candidate is processed by the parser. In our example, the parse accepts $b1f^\checkmark$. However, if the parser rejects the repair candidate c_1 , we add c_1 as a new counterexample and refine the hypothesis, producing the next approximation grammar G_2 .

This process continues until the repair is complete.

The repair produced is optimal with respect to the inferred grammar: no correction consistent with the grammar incurs fewer edits. As the grammar is iteratively refined to approximate the target format, successive repairs converge closer to the example data.

4 EVALUATION

How does β MAX compare with state-of-the-art? We propose the following research questions to investigate β MAX effectiveness.

4.1 Research Questions

Among parser-centric, format-agnostic repair techniques, ϵ REPAIR is state-of-the-art and uniquely handles records without identifiable waypoints, hence we use this as benchmark. Specification-based approaches such as RSR [35] are excluded as they are non-comparable. We also exclude $ddmax$ from the empirical comparison because its empty-start and waypoint assumptions fail for fixed-width formats such as dates, times, and network addresses, and because its deletion-only repair model cannot recover corruptions requiring insertion or replacement.

Given the limited information available to β MAX in comparison to ϵ REPAIR, the first question we ask is about whether data repair can actually repair a significant number of inputs.

RQ1: How many inputs can β MAX repair in comparison to ϵ REPAIR? An answer to this question can tell us whether grammar inference can form a useful tool for a data engineer.

Table 1: Subject regular expressions and average record lengths ($\mu|r|$) from 100 records.

Cat.	Regex	$\mu r $
Date	<code>^\d{4}-\d{2}-\d{2}\$</code>	10.00 $\pm$ 0.00
Time	<code>^\d{2}:\d{2}:\d{2}\$</code>	8.00 $\pm$ 0.00
ISBN	<code>^(?:\d[-]?){9}[\dX]\$</code>	13.00 $\pm$ 0.00
IPv4	<code>^(\d{1,3}\.){3}\d{1,3}\$</code>	13.60 $\pm$ 1.16
IPv6	<code>^([0-9a-fA-F]{1,4}){7}[0-9a-fA-F]{1,4}\$</code>	26.36 $\pm$ 3.00
URL	<code>^https?:\/\/(www\.)?[-a-zA-Z0-9@:%_\+~#&/=]*\$</code> <code>{1,256}\.[-a-zA-Z0-9()@%:_\+~#&/=]*\$</code> <code>\b([-a-zA-Z0-9()@%:_\+~#&/=]*)\$</code>	32.81 $\pm$ 12.89

The next question we ask is about the quality of data repair achieved in *conserving* original data.

RQ2: What is the quality of data repair by β MAX in comparison to ϵ REPAIR?

Finally, a data repair algorithm should be feasible to run for thousands of corrupt records. Hence, we ask:

RQ3: How does β MAX compare to other techniques in run-time performance and oracle calls required?

The details of experiments are given next.

Platform. Experiments were conducted on a Mac M2 Ultra with 192 GB RAM.

Research Protocol. We ran each repair technique and collected metrics, with a time limit of 300 seconds per record for both ϵ REPAIR and β MAX. Earley algorithm [6] was used for parsing.

4.2 Evaluation Subjects

To enable accurate assessment, we evaluated β MAX on six representative string-based input categories with regular expressions from regexlib.com: (1) date validating calendar dates, (2) time validating time stamps, (3) isbn validating ISBN identifiers, (4) ipv4 for IPv4 addresses, (5) ipv6 for IPv6 addresses, and (6) url for URLs.

Table 1 reports the regular expressions used for each category. We manually collected valid records for each of these categories. For each category, we curated a dataset of 100 valid examples. The average record lengths ($\mu|r|$) are shown in Table 1. We split these into two subsets randomly: 50 records for grammar inference (training) and 50 records for record repair (testing). In deployment, these positive examples are the valid records already present in the same column or field stream; no separately labeled corpus is required.

Single/Double/Triple Corruptions For each record, we induced corruptions uniformly at random from three operations: deletion, substitution, or insertion. We evaluated three corruption levels: one, two, and three point-corruptions. If a corruption did not result in a parse error, we resampled until a valid corrupt record was obtained. For each format, we produced 100 records for each of the 1-, 2-, and 3-corruption levels.

5 RESULTS

RQ1: How many corrupt records can be repaired by β MAX?

We compare the number of repaired files for β MAX and ϵ REPAIR (Table 2). The best values per row are bolded. We also provide the percentage of inputs repaired by each approach.

RQ2: What is the quality of data repair by β MAX and ϵ REPAIR? We collected all corrupt records that could be successfully repaired by each technique. For each repaired pair, we computed the *edit*

Table 2: Successfully repaired records across corruption levels (timeout = 300s, total records = 50 per subject per row).

	Subject	β MAX	β MAX %	ϵ REPAIR	ϵ REPAIR %
1-corruption	date	50	100%	15	30%
	ipv4	50	100%	40	80%
	ipv6	36	72%	35	70%
	isbn	49	98%	43	86%
	time	47	94%	14	28%
	url	28	56	37	74%
2-corruption	date	50	100%	13	26%
	ipv4	36	72%	37	74%
	ipv6	21	42%	32	64%
	isbn	48	96%	33	66%
	time	50	100%	9	18%
	url	38	76%	30	60%
3-corruption	date	48	96%	6	12%
	ipv4	50	100%	33	66%
	ipv6	23	46%	23	46%
	isbn	48	96%	17	34%
	time	48	96%	3	6%
	url	31	62%	6	12%
Total		751	83.4%	426	47%

distance between the corrupt record and the repaired record; results are summarized in Table 3. The standard deviation is given after $\pm$. The best values are marked in bold, with overall average last.

For a data engineer, the most important question in terms of data recovery is the amount of data that could be recovered from the given corrupted data. Table 4 captures the amount of data that was recovered from the corrupted data by each approach.

RQ3: How does β MAX compare to ϵ REPAIR in runtime performance and oracle calls required? We report average runtime in Table 5 and oracle calls in the repair phase in Table 6. β MAX averages 35.8 seconds per record compared to 55.9 seconds for ϵ REPAIR, a 1.6 $\times$ speedup. More significantly, β MAX invokes the oracle on average only 1.5 times per record compared to 41.5 times for ϵ REPAIR—a 27.7 $\times$ reduction. This advantage is especially important when oracle invocation is costly, for example when the validator is accessed over a network. A timeout sweep from 5 to 300 seconds showed that ϵ REPAIR’s lower repair rate is not primarily timeout-induced: ϵ REPAIR plateaued at 59.0% from 180 seconds onward, while β MAX repaired 74.7% of records at 5 seconds and 89.0% at 300 seconds. We additionally report the one-time *precompute* oracle cost of constructing the initial hypothesis DFA (including neighborhood exploration via mutation augmentation and cross-merge validation during state merging). This hypothesis DFA (and its merge history) is cached and reused across different inputs. When a candidate repair is rejected, β MAX refines the hypothesis DFA incrementally by replaying the recorded merge history under the expanded negative set, rather than re-running state merging from scratch. Therefore, the *precompute* oracle calls in Table 7 are amortized and incurred only once per subject. The detailed stage-wise breakdown of β MAX execution time is given in Table 8 and discussed further in Section 6.

6 DISCUSSION

A significant number of real-world, especially tabular data formats are validated by regular expressions [17]. In many cases, it is impossible to modify the parser to provide an error feedback as ϵ REPAIR requires. In such cases, one needs to rely on data-imputation from

Table 3: Levenshtein distances between corrupt and repaired data across corruption levels.

	Subject	βMAX	ϵREPAIR
1-corruption	date	1.66 ± 1.42	1.31 ± 0.46
	ipv4	5.00 ± 3.36	1.07 ± 0.26
	ipv6	2.56 ± 2.03	1.14 ± 0.35
	isbn	1.18 ± 0.39	1.58 ± 0.49
	time	1.53 ± 1.18	1.43 ± 0.49
	url	1.75 ± 1.77	1.05 ± 0.23
2-corruption	date	2.02 ± 0.68	1.94 ± 0.56
	ipv4	1.69 ± 0.62	1.65 ± 0.58
	ipv6	2.90 ± 2.22	1.81 ± 0.53
	isbn	1.90 ± 0.55	2.09 ± 0.62
	time	1.92 ± 0.34	1.86 ± 0.52
	url	3.32 ± 3.06	1.77 ± 0.42
3-corruption	date	2.85 ± 0.54	2.83 ± 0.37
	ipv4	6.64 ± 3.45	2.12 ± 0.69
	ipv6	3.87 ± 2.29	2.04 ± 0.69
	isbn	2.65 ± 0.63	3.00 ± 0.69
	time	3.02 ± 0.69	3.00 ± 0.00
	url	4.10 ± 3.40	2.50 ± 0.50
Average		2.79 ± 2.35	1.70 ± 0.71

Table 5: Average total runtime.

	Subject	βMAX	ϵREPAIR
1-corruption	date	1.56 ± 0.48	7.31 ± 10.48
	ipv4	50.24 ± 59.71	35.77 ± 49.92
	ipv6	148.99 ± 78.62	41.92 ± 77.32
	isbn	16.66 ± 38.98	91.82 ± 61.26
	time	1.09 ± 0.47	59.75 ± 83.45
	url	55.04 ± 65.95	52.10 ± 76.04
2-corruption	date	3.51 ± 0.37	12.34 ± 26.34
	ipv4	35.43 ± 72.20	48.34 ± 60.73
	ipv6	171.74 ± 77.60	45.24 ± 79.13
	isbn	21.23 ± 44.93	94.20 ± 58.51
	time	1.41 ± 0.10	104.78 ± 88.05
	url	50.20 ± 69.06	135.91 ± 91.28
3-corruption	date	1.85 ± 0.26	120.43 ± 75.93
	ipv4	90.49 ± 55.76	39.47 ± 45.69
	ipv6	128.41 ± 80.97	53.45 ± 78.91
	isbn	14.72 ± 33.95	115.58 ± 59.01
	time	1.40 ± 0.12	161.03 ± 93.51
	url	38.22 ± 54.43	77.02 ± 67.18
Overall avg.		35.80 ± 64.38	55.94 ± 70.19

available example records. βMAX shows that data-imputation from such available example records can be practical, and can even be faster than the state-of-the art baseline ϵREPAIR .

Our results from Table 3 show that βMAX achieves an average edit distance of 2.8 compared to 1.7 achieved by ϵREPAIR . Indeed,

Table 4: Data recovery between corrupt and repaired data across corruption levels.

	Subject	βMAX	ϵREPAIR
1-corruption	date	$91\% \pm 0\%$	$91\% \pm 0\%$
	ipv4	$91\% \pm 4\%$	$91\% \pm 3\%$
	ipv6	$97\% \pm 2\%$	$96\% \pm 0\%$
	isbn	$99\% \pm 2\%$	$92\% \pm 0\%$
	time	$89\% \pm 2\%$	$89\% \pm 1\%$
	url	$94\% \pm 4\%$	$97\% \pm 1\%$
2-corruption	date	$90\% \pm 5\%$	$89\% \pm 3\%$
	ipv4	$88\% \pm 6\%$	$89\% \pm 5\%$
	ipv6	$96\% \pm 2\%$	$94\% \pm 2\%$
	isbn	$98\% \pm 4\%$	$90\% \pm 4\%$
	time	$87\% \pm 7\%$	$86\% \pm 4\%$
	url	$91\% \pm 5\%$	$97\% \pm 3\%$
3-corruption	date	$87\% \pm 7\%$	$84\% \pm 5\%$
	ipv4	$87\% \pm 6\%$	$85\% \pm 5\%$
	ipv6	$96\% \pm 1\%$	$93\% \pm 2\%$
	isbn	$97\% \pm 4\%$	$84\% \pm 3\%$
	time	$81\% \pm 7\%$	$78\% \pm 2\%$
	url	$91\% \pm 5\%$	$93\% \pm 2\%$
Average		$92\% \pm 7\%$	$91\% \pm 5\%$

Table 6: Oracle calls in the repair phase.

	Subject	βMAX	ϵREPAIR
1-corruption	date	1.00 ± 0.00	7.00 ± 0.00
	ipv4	1.00 ± 0.00	16.48 ± 27.14
	ipv6	1.94 ± 1.00	23.92 ± 54.87
	isbn	2.20 ± 4.97	61.23 ± 46.38
	time	1.00 ± 0.00	7.00 ± 0.00
	url	1.64 ± 1.91	15.47 ± 30.68
2-corruption	date	1.00 ± 0.00	68.38 ± 45.87
	ipv4	3.57 ± 5.30	33.94 ± 42.83
	ipv6	1.81 ± 0.59	33.23 ± 60.02
	isbn	2.33 ± 3.78	71.23 ± 44.24
	time	1.00 ± 0.00	79.00 ± 64.94
	url	1.00 ± 0.00	103.08 ± 66.51
3-corruption	date	1.00 ± 0.00	93.50 ± 60.54
	ipv4	1.72 ± 0.45	28.76 ± 33.61
	ipv6	1.65 ± 0.63	40.35 ± 65.23
	isbn	1.92 ± 3.74	82.29 ± 42.86
	time	1.00 ± 0.00	113.67 ± 67.65
	url	1.00 ± 0.00	53.50 ± 47.21
Overall avg.		1.50 ± 2.32	41.51 ± 52.68

having access to accurate error feedback allows ϵREPAIR to identify precisely where to repair so as to minimize the number of edits. However, even without access to parse feedback—we only know whether the data is accepted or not— βMAX repairs come within one edit distance on average to that achieved by ϵREPAIR .

Table 7: β MAX precomputation oracle calls
Includes mutation augmentation and cross-merge validation.

Subject	#Oracle calls
date	1070
time	734
isbn	2751
ipv4	3903
url	854
ipv6	8566

Table 8: β MAX stage-wise cost across corruption levels (iterations and time breakdown; timeout = 300s).

	Sub.	G. Inf (sec)	#Oracle calls	Orc. time (sec)	G. Repair (sec)	Total time (sec)
1-corruption	date	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.88 $\pm$ 0.14	0.96 $\pm$ 0.14
	time	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.36 $\pm$ 0.06	0.43 $\pm$ 0.06
	url	0.38 $\pm$ 1.15	1.64 $\pm$ 1.91	0.03 $\pm$ 0.04	79.73 $\pm$ 65.47	80.23 $\pm$ 65.84
	isbn	3.70 $\pm$ 15.60	2.20 $\pm$ 4.97	0.04 $\pm$ 0.09	10.27 $\pm$ 21.21	14.08 $\pm$ 36.87
	ipv4	2.83 $\pm$ 13.87	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	17.44 $\pm$ 1.57	20.36 $\pm$ 14.15
	ipv6	7.49 $\pm$ 8.62	1.94 $\pm$ 1.00	0.04 $\pm$ 0.02	141.25 $\pm$ 69.16	148.87 $\pm$ 73.89
2-corruption	date	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.80 $\pm$ 0.22	1.89 $\pm$ 0.22
	time	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.32 $\pm$ 0.10	1.41 $\pm$ 0.10
	url	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	49.08 $\pm$ 69.04	50.20 $\pm$ 69.06
	isbn	7.53 $\pm$ 21.84	2.33 $\pm$ 3.78	0.04 $\pm$ 0.07	12.60 $\pm$ 23.07	21.23 $\pm$ 44.93
	ipv4	14.38 $\pm$ 31.85	3.57 $\pm$ 5.30	0.06 $\pm$ 0.10	19.93 $\pm$ 41.03	35.43 $\pm$ 72.20
	ipv6	15.82 $\pm$ 12.35	1.81 $\pm$ 0.59	0.03 $\pm$ 0.01	154.78 $\pm$ 68.25	171.74 $\pm$ 77.60
3-corruption	date	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.78 $\pm$ 0.26	1.85 $\pm$ 0.26
	time	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	0.34 $\pm$ 0.12	1.40 $\pm$ 0.12
	url	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.02 $\pm$ 0.00	37.10 $\pm$ 54.42	38.22 $\pm$ 54.43
	isbn	4.23 $\pm$ 15.97	1.92 $\pm$ 3.74	0.03 $\pm$ 0.07	9.41 $\pm$ 17.96	14.72 $\pm$ 33.95
	ipv4	74.53 $\pm$ 47.21	1.72 $\pm$ 0.45	0.03 $\pm$ 0.01	14.86 $\pm$ 8.76	90.49 $\pm$ 55.76
	ipv6	12.93 $\pm$ 13.20	1.65 $\pm$ 0.63	0.03 $\pm$ 0.01	114.37 $\pm$ 69.94	128.41 $\pm$ 80.97
Avg.		8.04 $\pm$ 24.92	1.50 $\pm$ 2.32	0.03 $\pm$ 0.04	27.01 $\pm$ 54.72	35.80 $\pm$ 64.38

G. Inf. = avg. grammar-inference time, Orc time = avg. time in oracle calls; G. Repair = avg. grammar-repair time, Total time = total time taken.

Critically, however, Table 4 shows that β MAX recovers 92% of the original data compared to 91% from ϵ REPAIR. This is important because the primary objective of a data engineer is to recover as much of the original data as possible.

Furthermore, β MAX relies on existing data for the inferred grammar, which means that the repairs suggested by β MAX are guided by existing data. In contrast, ϵ REPAIR repairs are guided exclusively by parser feedback. Hence, β MAX repairs will be closer to existing data records, and not unduly affected by parser idiosyncrasies.

In terms of performance, Table 5 suggests that β MAX has an average runtime of 35.8 seconds compared to 55.9 for ϵ REPAIR. That is, β MAX is 1.6 $\times$ faster than ϵ REPAIR in repairing data on average. Finally, we are leveraging an external program—the parser—for data-repair. Hence, it is instructive to look at how often this external program (the oracle) was invoked. Table 6 reports oracle calls in the repair phase. Our results show that, on average, β MAX invoked the oracle only once. That is, after β MAX inferred the base grammar from the provided examples, only a small correction was necessary on the grammar before the repaired input was accepted by the oracle. This is in contrast to ϵ REPAIR which required 41.5 oracle calls on average. This is not surprising, however, because each thread in ϵ REPAIR relies on oracle calls to provide guidance on what

Table 9: Ablation study on the number of initial positive seeds (K), with corruptions combined (1, 2, and 3).

K	Edit distance	Avg. Repair rate
50	2.79 $\pm$ 2.35	83.33%
25	2.47 $\pm$ 1.65	72.11%
12	2.80 $\pm$ 1.87	79.11%
6	3.98 $\pm$ 3.97	82.78%

Edit distances are computed only over successfully repaired cases.

next to do. We observe here that the time taken by oracle calls is dependent on the subject and can vary widely. In particular, the validating parser may even be over the network, which can make oracle calls very expensive. β MAX is therefore best suited to offline or batch-cleaning workflows where a small fraction of records fail validation and avoiding data loss is more important than sub-second latency. Repair attempts are independent across records after the initial grammar-inference step, so they can be parallelized when many failed records are available.

6.1 Ablation study of β MAX performance.

Why does β MAX require 35.8 seconds on average even though it only needs one oracle call? To investigate this, we decomposed the execution stages of β MAX. This is provided in Table 8. The table shows that the biggest chunk of computation is spent on *grammar-repair* (G. Repair). That is, the regular grammar learned from *grammar inference* is extended into a covered grammar, and this grammar is used to parse the corrupted string. There are several potential ways to parse the string, all of which needs to be explored concurrently, and the best one with least penalty chosen, which induces the computational overhead.

Impact of initial positive examples. One of the questions that can be asked is about the impact of the initial seed corpus. In Table 9 we document the impact of initial seeds used (K). The *edit-distance* column is the edit distance between corrupted and repaired strings. Our analysis reveals that $K = 50$ is the sweet-spot for the number of initial positive samples, which obtains the maximum number of repaired strings. When using valid examples more than this number, the initial automata produced becomes too complex, resulting in a much longer grammar-inference and grammar-repair time, reducing the success rate for larger samples. On the other hand, when using valid examples smaller than $K = 50$, the initial grammar inferred is rather simple, and multiple iterations are required to complete the repair. The edit distance is smaller at $K = 25$ because harder-to-repair corrupted strings were excluded from the successfully repaired set.

Impact of Neighborhood Exploration. Another question that we wanted to explore was about the impact of *neighborhood exploration*. How many new invalid strings should be constructed so that the repair quality is sufficient? To understand, we plotted the impact of different neighborhood (invalid) samples on repair quality. The result is given in Table 10.

Impact of Active Cross-Merge Validation A final question is on the impact of number of samples for cross-merge validation during state merging. The ablation study of 0 to 3 samples is provided in Table 11. Our results show that the maximum repair rate

is achieved under 1 cross-merge validation sample, with a small (0.04%) decrease in recovery compared to no cross-merge validation at all.

Table 10: Ablation study: impact of data augmentation with NE on repair performance (mutations combined).

#NE	Edit distance	Avg iterations	Repair rate
0	2.09 ± 1.22	6.69 ± 23.32	67.44%
20	2.23 ± 1.29	4.56 ± 17.57	73.56%
40	2.71 ± 2.91	7.27 ± 23.56	70.78%
60	2.79 ± 2.35	8.04 ± 24.92	83.33%
80	2.01 ± 1.17	8.40 ± 24.24	73.00%
100	2.27 ± 1.54	5.59 ± 19.96	74.67%

Table 11: Ablation study: impact of cross-merge validation (Samples) on repair performance (mutations combined).

Samples	Edit distance	Data recovery	Repair rate
0	3.45 ± 1.53	90.48% ± 6.25%	71.44%
1	2.49 ± 1.39	90.41% ± 6.73%	83.78%
2	2.47 ± 1.37	90.39% ± 6.80%	81.56%
3	2.54 ± 1.39	90.34% ± 6.70%	81.78%

The columns are as follows: #NE reports the number of invalid strings generated, *Edit-Distance* shows the average Levenshtein distance between corrupted and repaired strings, *Avg iterations* show the number of iterations necessary before the number of required invalid strings were generated, and *Repair rate* shows the number of successful repairs made.

Our analysis suggests that the repair rate peaks when the number of invalid strings is about 60. Hence, this is the number of invalid neighbors we used for empirical evaluation.

6.2 Data Imputation

How does our approach differ from regular-expression based structural repair methods such as RSR [35]? We note that each transition in the automaton we produce is based on at least one valid data record given. We only augment the set of invalid data records through mutation, and we intentionally do not attempt active grammar inference (e.g., with L^*). This means that any data record produced will have values that are closest to other records in the data set. For example, it is likely that a host name field may accept `ipv4` or `ipv6` records. However, if all existing data is based on domain names, it is unlikely that a given record is an `ipv4` record even if the closest valid correction is to make the data an `ipv4` record. If a precise regular expression or schema is available, specification-based repair such as RSR remains preferable. β MAX targets the complementary case where validation is embedded in ad hoc code, a compound matcher, or an inaccessible parser, and the only reliable interface is accept/reject feedback. As with numerical data imputation, the repaired value should be treated as a candidate that can be flagged for downstream review when semantic correctness matters; the alternative in many pipelines is discarding the record outright.

6.3 Applicability Beyond Regular Languages

How does β MAX fare against ϵ REPAIR when repairing complex data formats beyond regular expressions? We evaluated both on JSON, a context-free language, with results in Table 12 and Table 13. Even though β MAX approximates the JSON format using a regular grammar, it achieves superior data recovery compared to ϵ REPAIR across all corruption levels (95% vs 82% on average). The consistent data-recovery advantage suggests that regular-grammar approximation is a practical strategy even for context-free formats.

7 LIMITATIONS

Computational Complexity of Grammar Repair. The grammar-repair phase involves constructing and parsing with a covering grammar, which is inherently context-free. General context-free parsing has $O(n^3)$ time complexity with respect to input length. As shown in Table 8, the grammar-repair stage dominates execution time for longer inputs such as IPv6 and URL. While this overhead is acceptable for typical data-record lengths encountered in tabular data, it can become unwieldy for substantially larger inputs. Optimizations such as more efficient parsing strategies may be necessary for scaling to longer records.

Dependence on Initial Sample Quality. The performance of β MAX strongly depends on the representativeness of the initial valid samples. As demonstrated in our ablation study (Table 9), both repair rate and recovery quality suffer when insufficient or non-representative samples are provided. If the initial corpus does not adequately capture the variability of the target format, the inferred grammar may be either too specific—failing to generalize to valid corrupted inputs—or too permissive after state merging—producing repairs that diverge from the original data. This is not a problem in practice where tabular data often contain a large number of valid records. If otherwise, practitioners should ensure that the seed corpus reflects the expected data record diversity.

Regular Grammar Assumption. β MAX assumes that the data format accepted by the oracle can be approximated by a regular grammar. This assumption holds for many string-based formats commonly found in tabular data, including dates, times, identifiers, and network addresses. However, this assumption may not be valid for formats requiring context-free or context-sensitive grammars, such as nested JSON structures, XML documents, or programming language fragments. Similarly, binary formats such as TLV (Type-Length-Value) encodings, which encode structural constraints in field lengths, are not well-suited to our approach. Extending β MAX to handle such formats would require fundamentally different grammar inference and repair mechanisms.

Oracle Invocation Constraints. β MAX assumes that the oracle can be invoked programmatically and repeatedly at low cost. However, in certain practical settings, repeated oracle invocation may be infeasible, or prohibitively expensive. In such scenarios, a passive grammar inference approach with a more comprehensive initial corpus may be preferable.

Independent Record Repair. β MAX repairs each corrupted field independently. In tabular data, multiple records in the same column may be corrupted in similar ways, and exploiting such cross-record patterns could further guide repair. We leave joint repair across related records as future work.

Table 12: Levenshtein distances between corrupt and repaired data for JSON (timeout = 300s, total = 50 per row).

Corruption	βMAX	ϵREPAIR
1-corruption	2.12 ± 1.42	2.68 ± 2.80
2-corruption	3.49 ± 1.13	3.30 ± 2.30
3-corruption	3.94 ± 1.04	5.10 ± 2.87
Average	3.18 ± 1.20	3.69 ± 2.66

8 RELATED WORK

We next document some of the closely related research.

Constraint-Based Input Repair. Constraint-based input repair techniques restores invalid inputs by enforcing structural or semantic constraints. Such constraints may either be explicitly specified by developers or inferred automatically from a corpus of valid examples. Early work on data structure repair by Demsky and Rindard [13, 14] and subsequent extensions [47] focus on learning consistency invariants over data structures (e.g., pointer relationships or object graphs) and repairing violations to reestablish a consistent state. Related approaches in data cleaning and validation similarly rely on integrity constraints or rules learned from samples [36] or provided by users [16].

Grammar and Schema Inference. Grammar inference and grammar mining recover input specifications from examples or executions [9, 21, 27, 37]. Schema-inference techniques similarly infer reusable schemas for semi-structured data such as JSON collections [7]. These techniques are complementary to βMAX : their goal is to recover a reusable specification, whereas βMAX uses an inferred approximation as an intermediate artifact for repairing a particular rejected record under a black-box validator.

Specification-Less Black-Box Input Repair. Specification-less black-box input repair techniques treat the target program or parser purely as an oracle that classifies inputs as valid or invalid without assuming access to internal program state, source code, or grammar specifications. *ddmax* [31] was the first black-box approach designed for input repair. It frames repair as a maximization problem: What is the maximal valid string that can be recovered from the corrupted input string? While effective in many scenarios, *ddmax* is inherently limited by its deletion-only repair model and its assumption that a minimal valid base input exists, and assumption of waypoints. Finally, *ddmax* can also result in extreme data-loss at times when corruptions interact with the *ddmax* partitioning [38].

ϵREPAIR [38] improves upon *ddmax* by incorporating insertion operations and by exploiting richer forms of parser feedback, enabling it to repair inputs that require the addition or modification of symbols rather than simple removal. However, such accurate feedback is not guaranteed in several places where input repair may be needed. While ϵREPAIR can repair data conforming to regular expressions, it requires specially instrumented regular expression library *RE2* for this purpose. This library may not be usable in all circumstances. To our knowledge, βMAX is the only algorithm that can repair data that is described by regular expressions in blackbox settings without access to instrumented libraries.

White- and Gray-Box Input Repair. White- and gray-box input repair techniques leverage varying degrees of access to the

Table 13: Data recovery between corrupt and repaired data for JSON (timeout = 300s, total = 50 per row).

Corruption	βMAX	ϵREPAIR
1-corruption	$97\% \pm 6\%$	$88\% \pm 25\%$
2-corruption	$94\% \pm 7\%$	$85\% \pm 19\%$
3-corruption	$94\% \pm 7\%$	$72\% \pm 28\%$
Average	$95\% \pm 7\%$	$82\% \pm 24\%$

internal behavior of the program while processing the input. *Docovary* [33] uses symbolic execution to explore alternative execution paths and automatically modify corrupted documents so that the program avoids error states. Other techniques identify and analyze the specific regions of the input responsible for failures [3], enabling targeted modifications to those regions. Such approaches can produce precise repairs by understanding *why* an input fails, rather than merely observing *that* it fails. However, in contrast to βMAX , these methods require access to program internals, source code, or sophisticated analysis infrastructure, which may not be available or practical for arbitrary parsers or large collections of programs.

Parser-Directed Input Repair. Parser-directed input repair originates from compiler theory and focuses on recovering from syntax errors during parsing in order to continue processing the input. Classical techniques include minimum-distance error-correcting parsers [1] and heuristic-based recovery strategies for LL and LR parsers [4, 5, 8, 11]. Other work studies forward and backward recovery moves [10, 39]. Panic-mode recovery, which skips input symbols until a synchronization point is reached, is widely used in practice and surveyed in detail by Hammond and colleagues [22].

Modern parser generators such as ANTLR [43] incorporate sophisticated error recovery mechanisms, including token insertion, deletion, and replacement, to ensure that parsing can proceed even in the presence of errors. Recent work has revisited parser error recovery with a focus on improving diagnostic quality and minimizing unintended edits [15]. These approaches typically require access to an explicit grammar and prioritize continued parsing over faithful data recovery. As a result, the repaired input may be syntactically valid but semantically distant from the original data. In contrast, βMAX aims to compute minimal-edit repairs guided by parser feedback, without necessarily requiring an explicit grammar, and with an emphasis on maximally preserving the original input.

LLM-Based Input Repair. Recent work explores large language models (LLMs) for input repair and data cleaning [41, 52, 53]. LLM-based approaches are particularly effective for common and well-represented formats such as INI, JSON, and SExp, especially when similar data appears frequently in training corpora. By leveraging contextual understanding and learned patterns, LLMs can often propose plausible repairs for both syntactic and semantic errors.

Despite these advantages, our focus remains on algorithmic repair techniques: they offer deterministic, reproducible behavior, stronger correctness guarantees, and are free from hallucination or prompt sensitivity. They also integrate naturally into LLM pipelines as validation or minimization tools, supplying provably optimal solutions for downstream processing.

9 THREATS TO VALIDITY

External Validity. While we evaluated β MAX across seven data formats, this covers only a narrow slice of the formats encountered in practice. β MAX operates at the character level over an ASCII alphabet, so formats requiring Unicode or token-level representations fall outside its current scope. The corruption model used in our study may also not capture all real-world corruption patterns.

Internal Validity. Bugs in our implementation or inaccuracies in the baseline reimplementations could introduce bias. To reduce this risk we validated against known repair cases and relied on publicly available or carefully reconstructed baselines, though residual errors cannot be ruled out. All benchmarks were run on an Apple M2 Ultra workstation, so results on lower-powered hardware may differ. The 300-second per-repair time limit was fixed without a sensitivity study; tighter or looser budgets could shift both repair rate and edit-distance outcomes. Additionally, β MAX stops at the first accepted repair rather than exhaustively searching for the best one, which may occasionally globally suboptimal result.

Construct Validity. Repair quality is quantified through edit distance, data-recovery rate, and parser acceptance. Although these metrics are well-established, they reflect structural fidelity rather than semantic correctness or suitability for downstream use. No human or domain-expert judgement was collected, which constrains how confidently the results can be interpreted in application contexts. Future work should address these gaps through user studies and broader downstream evaluations.

10 CONCLUSION

This paper presents β MAX, a novel specification-free algorithm for repairing corrupted data records, producing repairs optimal with respect to the provided example data. By overcoming the limitations of both specification-dependent methods and existing specification-free techniques, β MAX enables more flexible and effective structural recovery of malformed inputs.

Experimental results demonstrate clear improvements over the state-of-the-art ϵ REPAIR, achieving $1.77\times$ repair ability, while reducing oracle calls $27.7\times$ compared to ϵ REPAIR, and maintaining the quality of data recovery at 92%. These findings establish β MAX as a robust and practical solution for data repair in real-world data-processing pipelines, particularly in environments where formal specifications or strict parser assumptions are unavailable but sufficient examples exist to guide the repair process.

DATA AVAILABILITY

All data for this research is available at:

<https://doi.org/10.5281/zenodo.19122956>

REFERENCES

- [1] Alfred V Aho and Thomas G Peterson. 1972. A minimum distance error-correcting parser for context-free languages. *SIAM J. Comput.* 1, 4 (1972), 305–312.
- [2] Alfred V Aho and Jeffrey D Ullman. 1971. The care and feeding of LR(k) grammars. In *ACM Symp. Theory Comput. (STOC)*. 159–170.
- [3] Paul Eric Ammann and John C Knight. 1988. Data diversity: An approach to software fault tolerance. *IEEE Trans. Comput.* 4 (1988), 418–425.
- [4] S. O. Anderson and Roland Carl Backhouse. 1981. Locally least-cost error recovery in Earley’s algorithm. *ACM Trans. Program. Lang. Syst.* 3, 3 (1981), 318–347.
- [5] S. O. Anderson, Roland Carl Backhouse, E. H. Bugge, and C. P. Stirling. 1983. An assessment of locally least-cost error recovery. *Comput. J.* 26, 1 (1983), 15–24.
- [6] John Aycock and R Nigel Horspool. 2002. Practical Earley parsing. *Comput. J.* 45, 6 (2002), 620–630.
- [7] Mohamed-Amine Baazizi, Dario Colazzo, Giorgio Ghelli, and Carlo Sartiani. 2019. Parametric schema inference for massive JSON datasets. *The VLDB Journal* 28, 4 (2019), 497–521. <https://doi.org/10.1007/s00778-018-0532-7>
- [8] Roland C Backhouse. 1979. *Syntax of Programming Languages: Theory and Practice*. Prentice-Hall.
- [9] Miguel Bugalho and Arlindo L Oliveira. 2005. Inference of regular languages using state merging algorithms with search. *Pattern Recognition* 38, 9 (2005), 1457–1467.
- [10] Michael Burke and Gerald A. Fisher. 1982. A practical method for syntactic error diagnosis and recovery. *ACM SIGPLAN Not.* 17, 6 (1982), 67–78.
- [11] Carl Cerecke. 2003. *Locally least-cost error repair in LR parsers*. Ph. D. Dissertation. University of Canterbury.
- [12] Data Core. 2021. Combatting Data Corruption in the Digital Age. <https://www.datacore.com/glossary/data-corruption/>.
- [13] Brian Demsky and Martin Rinard. 2003. Automatic Detection and Repair of Errors in Data Structures. *ACM SIGPLAN Not.* 38, 11 (2003), 78–95.
- [14] Brian Demsky and Martin Rinard. 2005. Data structure repair using goal-directed reasoning. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. 176–185. <https://doi.org/10.1109/ICSE.2005.1553560>
- [15] Lukas Diekmann and Laurence Tratt. 2020. Don’t Panic! Better, Fewer, Syntax Errors for LR Parsers. In *Proc. Eur. Conf. Object-Oriented Program. (ECOOP) (LIPIcs, Vol. 166)*. Schloss Dagstuhl, 6:1–6:32.
- [16] Bassem Elkarablieh and Sarfraz Khurshid. 2008. Juzi: A tool for repairing complex data structures. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. ACM, 855–858.
- [17] Benjamin Eriksson, Amanda Stjerna, Riccardo De Masellis, Philipp Rüemmer, and Andrei Sabelfeld. 2023. Black Ostrich: Web Application Scanning with String Solvers. In *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*. 549–563.
- [18] Charles Fischer, Bernard Dion, and Jon Mauney. 1979. *A locally least-cost LR-error corrector*. Technical Report. University of Wisconsin-Madison Department of Computer Sciences.
- [19] Charles N Fischer and Jon Mauney. 1992. A simple, fast, and effective LL(1) error repair algorithm. *Acta Informatica* 29, 2 (1992), 109–120.
- [20] Marc Fisher and Gregg Rothermel. 2005. The EUSES spreadsheet corpus: a shared resource for supporting experimentation with spreadsheet dependability mechanisms. In *Workshop on End-User Software Engineering*.
- [21] Rahul Gopinath, Björn Mathis, Matthias Höschle, Alexander Kampmann, and Andreas Zeller. 2018. Sample-Free Learning of Input Grammars for Comprehensive Software Fuzzing. *arXiv preprint arXiv:1810.08289* (2018).
- [22] K Hammond and Victor J. Rayward-Smith. 1984. A survey on syntactic error recovery and repair. *Computer Languages* 9, 1 (1984), 51–67.
- [23] K. Harrenstien, M.K. Stahl, and E.J. Feinler. 1985. DoD Internet Host Table Specification. <https://www.rfc-editor.org/rfc/rfc952>.
- [24] Joseph M Hellerstein. 2013. *Quantitative data cleaning for large databases*. <https://dsf.berkeley.edu/jmh/papers/cleaning-unece.pdf>
- [25] Mauricio A Hernández and Salvatore J Stolfo. 1998. Real-world data is dirty: Data cleansing and the merge/purge problem. *Data Mining and Knowledge Discovery* 2 (1998), 9–37.
- [26] Peter H Hochschild, Paul Turner, Jeffrey C Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E Culler, and Amin Vahdat. 2021. Cores that don’t count. In *Workshop on Hot Topics in Operating Systems (HotOS)*.
- [27] Matthias Höschle and Andreas Zeller. 2017. Mining Input Grammars with AUTOGram. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. Companion (ICSE-C)*. IEEE, 31–34. <https://doi.org/10.1109/ICSE-C.2017.14>
- [28] ISO. 2019. *ISO 8601*. https://en.wikipedia.org/wiki/ISO_8601
- [29] Mujib Jimoh. 2023. *Date Formats: A Rare Headache in the Construction of Contractual Documents*. SSRN.
- [30] Ik-Soon Kim and Kwangkeun Yi. 2010. LR error repair using the A* algorithm. *Acta Informatica* 47, 3 (2010), 179–207.
- [31] Lukas Kirschner, Ezekiel Soremekun, and Andreas Zeller. 2020. Debugging Inputs. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. 75–86.
- [32] Marcin Kozak, Wojtek Krzanowski, Izabela Cichocka, and James Hartley. 2015. The effects of data input errors on subsequent statistical inference. *Journal of Applied Statistics* 42, 9 (2015), 2030–2037.
- [33] Tomasz Kuchta, Cristian Cadar, Miguel Castro, and Manuel Costa. 2014. Docoverly: Toward Generic Automatic Document Recovery. In *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. 563–574.
- [34] Kevin J. Lang, Barak A. Pearlmutter, and Rodney A. Price. 1998. Results of the Abbingdo One DFA Learning Competition and a New Evidence-Driven State Merging Algorithm. In *Grammatical Inference (Lecture Notes in Computer Science, Vol. 1433)*. Springer, 1–12. <https://doi.org/10.1007/BFb0054059>
- [35] Zeyu Li, Hongzhi Wang, Wei Shao, Jianzhong Li, and Hong Gao. 2016. Repairing data through regular expressions. *Proc. VLDB Endow.* 9, 5 (2016), 432–443.
- [36] Fan Long, Vijay Ganesh, Michael Carbin, Stelios Sidiropoulos, and Martin Rinard. 2012. Automatic Input Rectification. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. IEEE Press, 80–90.

- [37] Damián López and Pedro García. 2016. On the inference of finite state automata from positive and negative data. *Topics in Grammatical Inference* (2016), 73–112.
- [38] Zijian Luo, Lukas Kirschner, Ezekiel Soremekun, and Rahul Gopinath. 2025. Automatic Data Repair without Format Specifications. In *Proc. IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE)*.
- [39] Jon Mauney and Charles N. Fischer. 1982. A forward move algorithm for LL and LR parsers. *ACM SIGPLAN Not.* 17, 6 (1982), 79–87.
- [40] Mashaal Musleh, Mourad Ouzzani, Nan Tang, and AnHai Doan. 2020. CoClean: Collaborative Data Cleaning. In *Proc. ACM SIGMOD Int. Conf. Manag. Data.* 2757–2760.
- [41] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, and Jianwei Yin. 2024. IterClean: An Iterative Data Cleaning Framework with Large Language Models. In *ACM Turing Award Celebration Conference — China (ACM-TURC)*. 100–105.
- [42] José Oncina and Pedro García. 1992. Identifying regular languages in polynomial time. In *Advances in Structural and Syntactic Pattern Recognition*. World Scientific, 99–108.
- [43] Terence Parr and Kathleen Fisher. 2011. LL(*): The foundation of the ANTLR parser generator. *ACM SIGPLAN Not.* 46, 6 (2011), 425–436.
- [44] Stephen G. Powell, Kenneth R. Baker, and Barry Lawson. 2008. A critical review of the literature on spreadsheet errors. *Decision Support Systems* (2008).
- [45] S. Rajasekaran and M. Nicolae. 2016. An Error Correcting Parser for Context-Free Grammars that Takes Less Than Cubic Time. In *Proc. Int. Conf. Lang. Autom. Theory Appl. (LATA) (Lecture Notes in Computer Science, Vol. 9618)*. Springer, 533–546.
- [46] Fakhitah Ridzuan and Wan Mohd Nazmee Wan Zainon. 2019. A review on data cleansing methods for big data. *Procedia Computer Science* 161 (2019), 731–738. <https://doi.org/10.1016/j.procs.2019.11.177>
- [47] Martin C. Rinard. 2007. Living in the Comfort Zone. In *Proc. ACM Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*. ACM, 611–622.
- [48] Christopher Scaffidi, Brad Myers, and Mary Shaw. 2008. Topes. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. IEEE, 1–10.
- [49] Christopher Scaffidi and Mary Shaw. 2008. Accommodating data heterogeneity in ULS systems. In *Workshop on Ultra-Large-Scale Software-Intensive Systems*. 15–18.
- [50] Charles Cresson Wood and William W. Banks. 1993. Human error: an overlooked but significant information security problem. *Computers & Security* 12, 1 (1993).
- [51] Mark Woodard, Sahra Sedigh Sarvestani, and Ali Hurson. 2015. A Survey of Research on Data Corruption in Cyber-Physical Critical Infrastructure Systems. *Advances in Computers* (2015).
- [52] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. 2025. Aligning the Objective of LLM-based Program Repair. In *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*.
- [53] Shuo Zhang, Zezhou Huang, and Eugene Wu. 2024. Data cleaning using large language models. *arXiv preprint arXiv:2410.15547* (2024).

Received 2026-03-26; accepted 2026-06-18